AWS

VPC Peering

Connect two VPCs. Prove private connectivity.

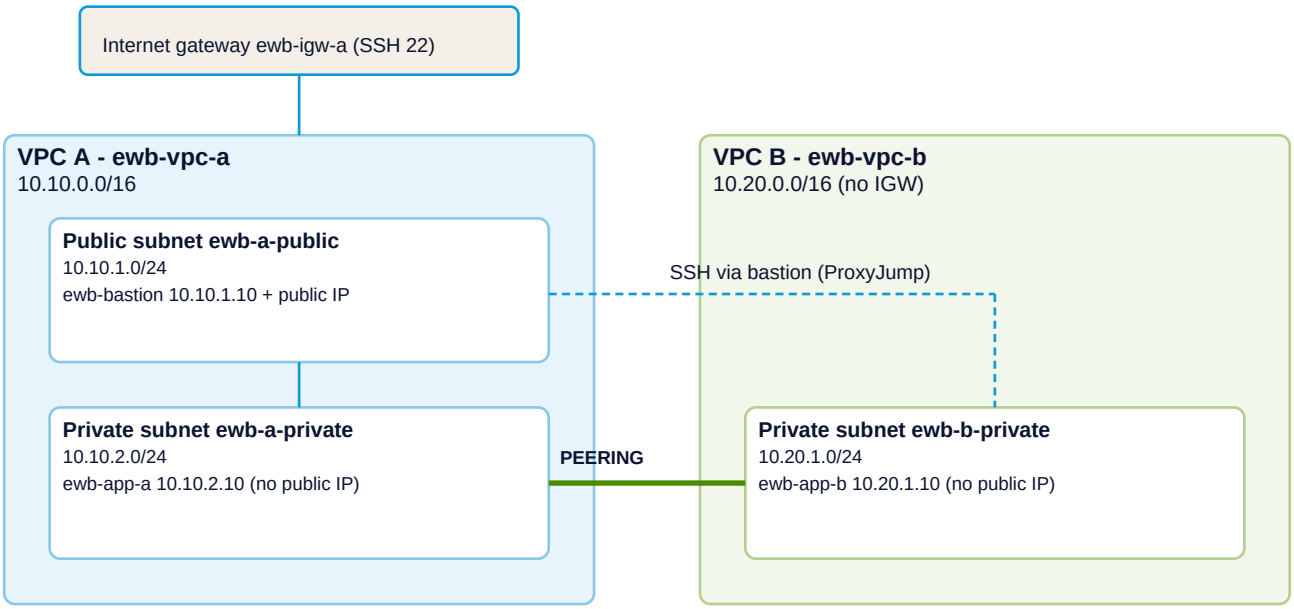
EWB Courses | AWS Architect Associate | Project 01 - Student Project Report

Name	Ashwani Kumar Pandey
Batch	AWS INTERNSHIP
Project	AWS VPC Peering with bastion host (Project 01)
Region	eu-north-1 (Europe - Stockholm)
Completion date	01-11-2026
Method	Built with AWS CLI scripts (repo on GitHub), tested over SSH; evidence captured from real execution

Mission. Two application servers (App-A in VPC A, App-B in VPC B) must stay private, yet exchange traffic over private IPv4 addresses. A VPC peering connection links the VPCs and a single public bastion in VPC A is used for administration with SSH ProxyJump. No NAT gateway and no internet gateway on VPC B.

Contents: 1 Architecture and inventory - 2 Subnets, internet gateway and route tables - 3 Peering and security groups - 4 Test evidence - 5 Controlled failure and recovery - 6 Forward and return path - 7 Viva answers - 8 Troubleshooting log - 9 Cleanup evidence - 10 What I learned

1. Architecture and resource inventory

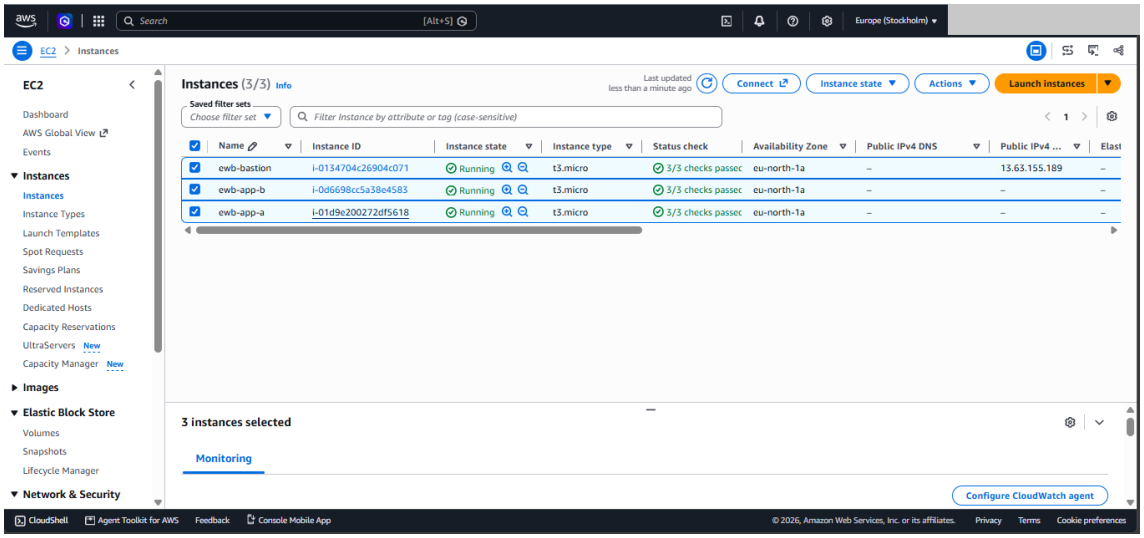


Administration path: laptop > bastion > App-A or App-B (SSH ProxyJump, the private key never leaves the laptop). Project traffic: App-A and App-B over private IPs through the peering connection. VPC B has no internet access.

Actual resource IDs (from evidence/01 and 03)

Resource	ID	CIDR / IP
ewb-vpc-a	vpc-02e3c533a27e66469	10.10.0.0/16
ewb-vpc-b	vpc-064c68d8e5ce55f0e	10.20.0.0/16
ewb-a-public	subnet-0cbacf036a1a2a916	10.10.1.0/24
ewb-a-private	subnet-0528dc19075bc718b	10.10.2.0/24
ewb-b-private	subnet-096a1a5a7611e5fd7	10.20.1.0/24
ewb-igw-a (attached to VPC A)	igw-0f55d0b76db861b40	-
ewb-rt-a-public	rtb-004800d9cdb38d6e0	assoc. ewb-a-public
ewb-rt-a-private	rtb-03a3ed229105ca173	assoc. ewb-a-private
ewb-rt-b-private	rtb-0db2f3008481e1051	assoc. ewb-b-private
ewb-a-b-peer (peering)	pcx-04a643b9500fec011	VPC A (requester) - VPC B (accepter)
ewb-bastion	i-0134704c26904c071	10.10.1.10 + public 13.63.155.189
ewb-app-a	i-01d9e200272df5618	10.10.2.10 (no public IP)
ewb-app-b	i-0d6698cc5a38e4583	10.20.1.10 (no public IP)

All resources are tagged **Project = EWB-VPC-Peering** and **Student = Ashwani Kumar Pandey**. Instances: Amazon Linux 2023, t3.micro, 8 GB gp3 root volume with delete-on-termination, key pair ewb-lab, IMDSv2 required.



Screenshot 1 - EC2 instances (eu-north-1): bastion has a public IPv4; App-A and App-B show none. All 3/3 status checks passed.

2. Subnets, internet gateway and route tables

The public subnet is explicitly associated with **ewb-rt-a-public** (default route to the internet gateway). Both private subnets are explicitly associated with their own route tables and have no internet route. Auto-assign public IPv4 is disabled on all subnets; only the bastion got a public IP at launch.

Before peering (Checkpoint 01, evidence/01)

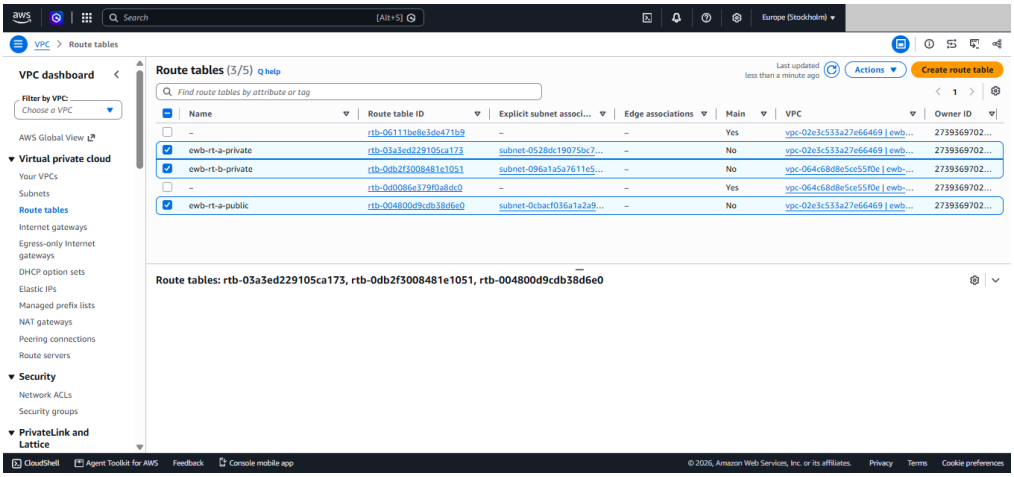
```
Subnets:  ewb-a-private 10.10.2.0/24 | ewb-b-private 10.20.1.0/24 | ewb-a-public 10.10.1.0/24  (MapPublicIpOnLaunch = False)

ewb-rt-a-private  rtb-03a3ed229105ca173  subnet-0528dc19075bc718b   10.10.0.0/16 local
ewb-rt-b-private  rtb-0db2f3008481e1051  subnet-096a1a5a7611e5fd7   10.20.0.0/16 local
ewb-rt-a-public   rtb-004800d9cdb38d6e0  subnet-0cbacf036a1a2a916   10.10.0.0/16 local | 0.0.0.0/0 igw-0f55d0b76db861b40
```

After peering (Checkpoint 04, evidence/03)

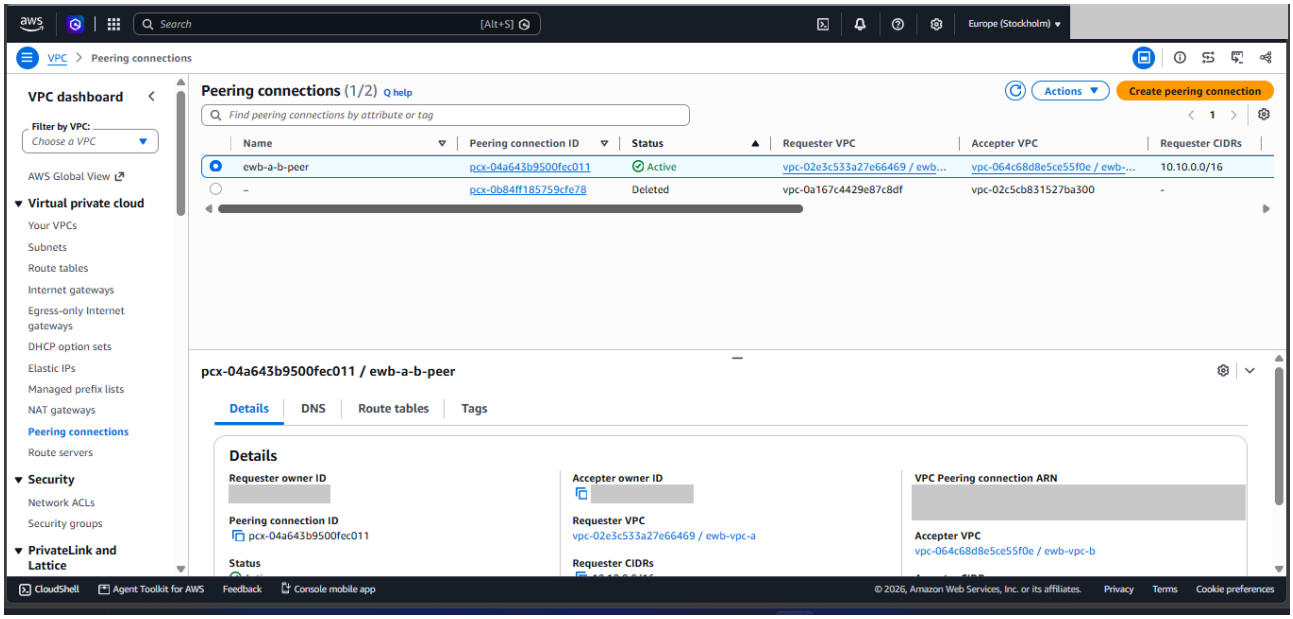
Route table / subnet	Destination	Target	State
ewb-rt-a-private / ewb-a-private	10.10.0.0/16	local	active
	10.20.0.0/16	pcx-04a643b9500fec011	active
ewb-rt-b-private / ewb-b-private	10.20.0.0/16	local	active
	10.10.0.0/16	pcx-04a643b9500fec011	active
ewb-rt-a-public / ewb-a-public	10.10.0.0/16	local	active
	10.20.0.0/16	pcx-04a643b9500fec011	active
	0.0.0.0/0	igw-0f55d0b76db861b40	active

The public subnet also needs the peering route because ProxyJump makes the bastion open its own SSH connection to App-B. The more specific 10.20.0.0/16 route wins over 0.0.0.0/0.



Screenshot 2 - Route tables (VPC console): the three custom tables each show an explicit subnet association.

3. Peering connection and security groups



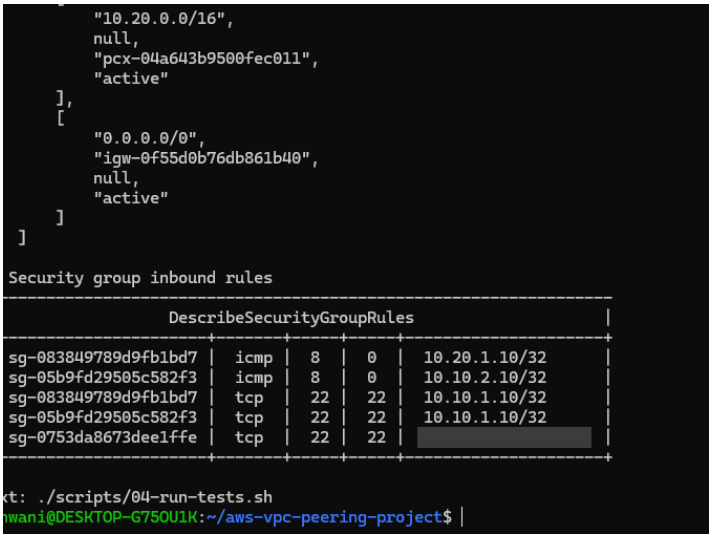
Screenshot 3 - ewb-a-b-peer (pcx-04a643b9500fec011) is Active; requester ewb-vpc-a, accepter ewb-vpc-b (account IDs masked).

The request was created from VPC A and accepted for VPC B (the accept step is required even inside one account). Active peering alone does not make ping or SSH work: routes on both sides and security-group rules are also needed.

Inbound rules (evidence/03; outbound left as default allow-all)

Security group	Type / port	Source
ewb-sg-bastion (VPC A)	SSH / TCP 22	my public IPv4 /32 (masked)
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a (VPC A)	ICMP echo request (type 8, code 0)	10.20.1.10/32 (App-B)
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b (VPC B)	ICMP echo request (type 8, code 0)	10.10.2.10/32 (App-A)

No rule opens SSH or ICMP to 0.0.0.0/0. Network ACLs are the default allow-all.



Screenshot 4 - Terminal output of the inbound rules and the active peering routes (my public IP masked).

4. Test evidence

All outputs below are real, copied from the evidence files produced by the test scripts (SSH post-quantum notice lines were removed for readability; they are only an OpenSSH client notice).

Test 01 - Baseline before peering (App-A pings App-B): FAILS as expected

```
# BASELINE (before peering) 2026-11-01T09:32:40+00:00
## ssh ewb-bastion    -> ip-10-10-1-10.eu-north-1.compute.internal / 10.10.1.10
## ssh ewb-app-a      -> ip-10-10-2-10.eu-north-1.compute.internal / 10.10.2.10
## App-A -> App-B ping
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3143ms
```



Screenshot 5 - Baseline: bastion and App-A reachable over local routes, App-A to App-B is 100% packet loss before peering.

Test 02 - Forward path, App-A to App-B (after routes)

```
ip-10-10-2-10.eu-north-1.compute.internal   10.10.2.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.432 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.184 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.195 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.188 ms
4 packets transmitted, 4 received, 0% packet loss, time 3091ms
rtt min/avg/max/mdev = 0.184/0.249/0.432/0.105 ms
```

Test 03 - Reverse path, App-B to App-A

```
ip-10-20-1-10.eu-north-1.compute.internal   10.20.1.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=127 time=0.179 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=127 time=0.194 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=127 time=0.161 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=127 time=0.197 ms
4 packets transmitted, 4 received, 0% packet loss, time 3131ms
rtt min/avg/max/mdev = 0.161/0.182/0.197/0.014 ms
```

Test 04 - Private SSH to App-B through the bastion (ssh ewb-app-b, ProxyJump)

```
Logged in to App-B via bastion
ip-10-20-1-10.eu-north-1.compute.internal
10.20.1.10
ec2-user
```

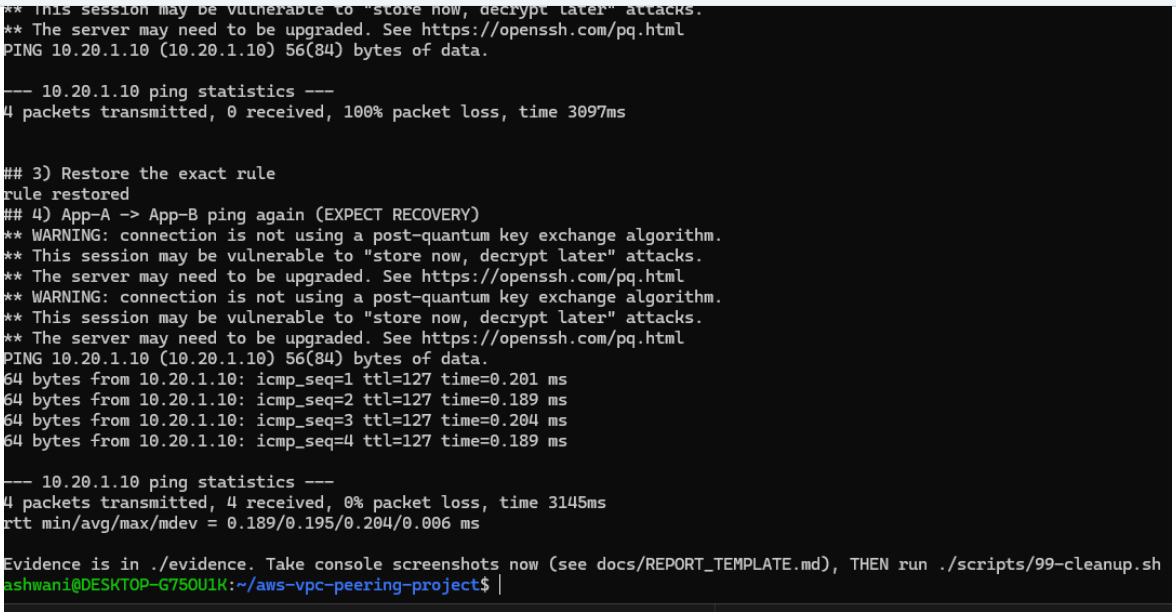
Test 05 - Isolation: no public IPv4 on App-A and App-B

```
|  ewb-app-b|  i-0d6698cc5a38e4583  |  10.20.1.10 |  None  |
|  ewb-app-a|  i-01d9e200272df5618  |  10.10.2.10 |  None  |
```

5. Controlled failure and recovery

I removed **only** the inbound ICMP echo-request rule (source 10.10.2.10/32) from **ewb-sg-app-b**, repeated the forward ping, restored the exact rule, and repeated it again. The SSH rule stayed in place.

```
# CONTROLLED FAILURE 2026-11-01T09:36:16+00:00
## 1) Remove ICMP Echo rule (10.10.2.10/32) from ewb-sg-app-b      -> rule removed
## 2) App-A -> App-B ping (EXPECT FAILURE)
4 packets transmitted, 0 received, 100% packet loss, time 3097ms
## 3) Restore the exact rule                                        -> rule restored
## 4) App-A -> App-B ping again (EXPECT RECOVERY)
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.201 ms  ... seq=4 time=0.189 ms
4 packets transmitted, 4 received, 0% packet loss, time 3145ms
```



Screenshot 6 - Rule restored and ping recovered to 0% packet loss (the 100% loss step is shown above).

Explanation. Peering and both route tables were still correct, yet ping failed as soon as the destination security group stopped allowing ICMP echo requests from App-A. When the same rule was restored, replies resumed immediately. So peering and routes provide a path, but security groups decide whether traffic on that path is allowed.

6. Forward path and return path

Forward: App-A (10.10.2.10) looks up ewb-rt-a-private, finds 10.20.0.0/16 -> pcx-04a643b9500fec011, crosses the peering connection into VPC B and reaches App-B (10.20.1.10). ewb-sg-app-b allows ICMP echo from 10.10.2.10/32.
Return: App-B replies to 10.10.2.10, looks up ewb-rt-b-private, finds 10.10.0.0/16 -> pcx-04a643b9500fec011 and sends the reply back to VPC A. Security groups are stateful, so the reply is allowed without an extra inbound rule. Without the return route the replies would be lost.
Administration path: laptop -> internet gateway -> bastion (10.10.1.10) -> App-A directly (local route) or App-B (public-subnet table 10.20.0.0/16 -> pcx). App-B sees the bastion private IP 10.10.1.10 as the SSH source.

7. Viva answers

1. Why is the return route needed? Peering does not route automatically. Replies from App-B need a route to 10.10.0.0/16 through the peering connection. A one-way route means requests arrive but replies cannot return.
2. Why is the App-B SSH source the bastion private IP? With ProxyJump the bastion opens a new SSH connection to App-B, so App-B sees 10.10.1.10, not my laptop.
3. Why does Active peering not guarantee a successful ping? Active only means the link exists. Route tables on both sides and security-group (and NACL) rules must also allow the traffic. My controlled failure showed this.
4. Can VPC B use VPC A's internet gateway over peering? No. Peering is one-to-one and non-transitive and does not provide edge-to-edge routing through an internet gateway. VPC B has no internet access.
Extra - why must CIDRs not overlap? 10.10.0.0/16 and 10.20.0.0/16 are distinct. With overlapping VPC CIDRs a peering connection cannot be created, and different subnet ranges alone do not fix it.

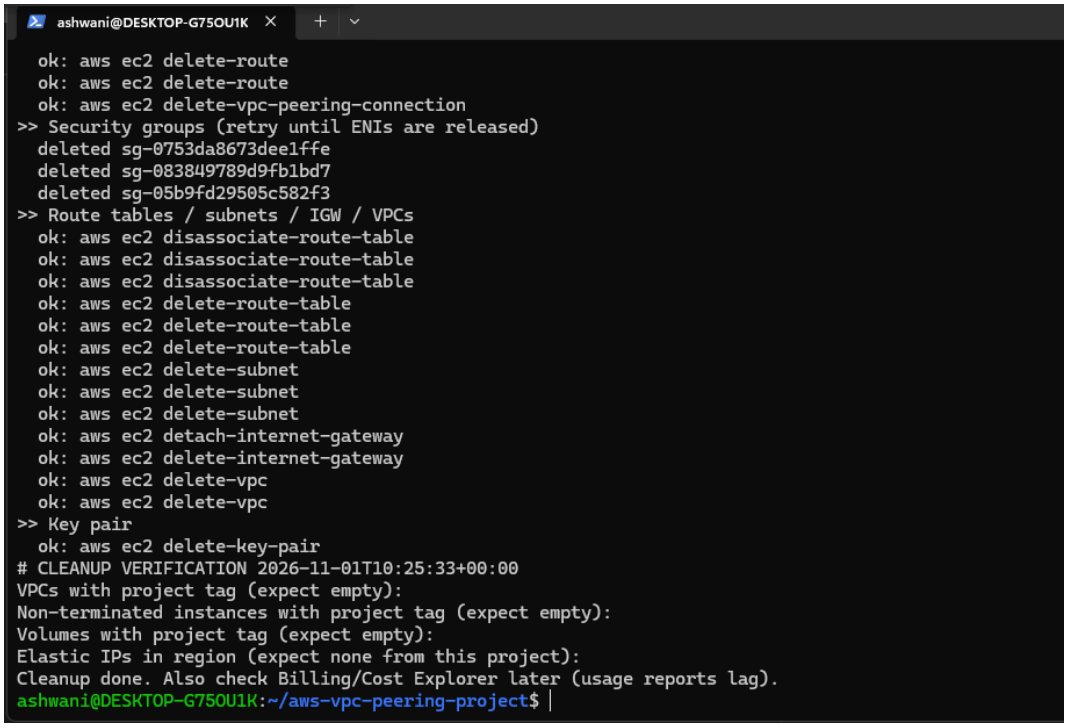
8. Troubleshooting log (from my first manual attempt)

Symptom	Root cause	Fix
Bastion SSH timed out	Public subnet was not explicitly associated with the route table that has the internet-gateway route	Explicit association with ewb-rt-a-public; auto-assign public IP for the bastion
ssh ewb-app-a: channel 0 open failed	App-A security group did not allow SSH from the bastion private IP	SSH rule from 10.10.1.10/32 (later tightened from a /16)
REMOTE HOST IDENTIFICATION HAS CHANGED	Re-created instances reused the same private IP with new host keys	Removed the stale known_hosts entries
Host key prompt aborted through ProxyJump on PowerShell	PowerShell swallowed the interactive prompt	Used WSL and StrictHostKeyChecking accept-new
Ping A to B showed 100% loss with peering + routes in place	ICMP echo-request rule was missing on the destination security group	Added the /32 ICMP rule on both app security groups

9. Cleanup evidence

Evidence was saved first. Then the cleanup script terminated the three instances, deleted the peer routes and ewb-a-b-peer, the security groups, route tables, subnets, the internet gateway, both VPCs and the key pair.

```
# CLEANUP VERIFICATION 2026-11-01T10:25:33+00:00
VPCs with project tag (expect empty):
Non-terminated instances with project tag (expect empty):
Volumes with project tag (expect empty):
Elastic IPs in region (expect none from this project):
```



Screenshot 7 - Cleanup run: every resource deleted and the verification lists are empty.

The access key I used for the CLI was deleted after the lab. Billing was checked for leftover chargeable resources (usage reports can lag).

10. What I learned

- VPC peering is one-to-one and non-transitive, and the VPC CIDR ranges must not overlap.
- A working peering needs three things together: an Active connection, routes on both sides (including the bastion subnet for ProxyJump), and matching security-group rules.
- Explicit route-table associations matter: an unassociated subnet silently falls back to the main route table.
- Security groups are stateful; routing is not. That is why the return route is required but a return security rule is not.
- Testing a failure on purpose (removing one rule) and watching it recover proves the cause of a problem, not only that things work.
- Bastion + ProxyJump keeps the private key on my laptop and the app servers off the public internet. Cleaning up the same day avoids charges for EC2, EBS and public IPv4.

Repository: github.com/<my-username>/aws-vpc-peering-project (scripts, README and raw evidence files).